NIGHTFLOW VIP

ENTERPRISE CLOUD ARCHITECTURE · OPERATIONS & SYSTEMS MANUAL

MANUAL TÉCNICO DE SISTEMAS

Módulo Sala, Presencia & Concierge en Tiempo Real

Especificación de Arquitectura Frontend React 19, Motor Cuádruple de Suscripción Firestore, Máquina de Estados de Presencia (AGENTS §6), Holds Criptográficos (AGENTS §7) y Protocolo de Reingreso Passkey / WebAuthn.

FRAMEWORK	BASE DE DATOS	CRIPTOGRAFÍA	MODO SPARKS
React 19 + Vite 8	**Cloud Firestore**	**Ed25519 / KMS**	**Render Híbrido**

SECCIÓN 1 Topología del Frontend y Jerarquía de Componentes

El módulo de operaciones en sala reside en el namespace de administración y se expone mediante la ruta de control operativo. Su orquestación está contenida en el componente central:

```
src/components/club/dashboard/FloorOpsView.tsx
├── InteractiveFloorPlan.tsx      <-- Renderizador SVG/Canvas cenital + tableStateOverrides
├── OperatingTablesList.tsx       <-- Vista tabular reactiva + acción de liquidación (Settle Modal)
├── ConciergeQueueList            <-- Cola de peticiones en vivo (Filtros: ALL, PENDING,
IN_PROGRESS, DONE)
└── TableOperationalDrawer        <-- Inspección profunda de mesa (comanda, cliente, aforo,
acciones)
```

1.1 Control de Acceso y RBAC por Plan El componente verifica la permisología en dos niveles antes del renderizado:

1. **Plan Comercial del Establecimiento:** `isDashboardTabAllowed('FLOOR_OPS', activeClub, activePlan)` valida que el negocio posea un plan con soporte para gestión de piso (**Scale/VIP Enterprise**).
2. **Capacidades de Staff:** `canStaffSettleTab(session?.staffRole, session?.isSuperAdmin)` restringe la liquidación de cuentas exclusivamente a los roles `admin`, `manager` y `cashier`. El personal de puerta o mozos carece de acceso mutacional a los cobros.

1.2 Conmutador de Modo Operativo Dual El estado `opsMode` soporta dos modalidades complementarias:

- `MAP` (**Sala en Vivo**): Prioriza la vista cenital bidimensional interactiva, mostrando las mesas georreferenciadas con sus halos de presencia y panel lateral de métricas de sala.
- `CONCIERGE` (**Piso & Concierge**): Prioriza la cola de tickets de atención a mesas ordenados por severidad y tiempo de espera, conservando el plano en miniatura para ubicación espacial del corredor.

> **DIRECTRIZ AGENTS §10 — CERO SIMULACIONES**
>
> Queda erradicada cualquier simulación de roles o manipulación directa de documentos desde el frontend. Toda mutación sensible (cierre de cuentas, asignación de tickets de concierge o anulación de holds) se ejecuta a través de llamadas invocables al backend (`callFunctions`) con ID Tokens verificados criptográficamente.

SECCIÓN 2 Motor en Tiempo Real y Cuádruple Suscripción Firestore

Para evitar desincronizaciones entre lo que ocurre en la puerta, en la pasarela web y en la barra, el servicio `operationsService.ts` (`subscribeOperations`) implementa una suscripción cuádruple simultánea mediante listeners `onSnapshot` de Cloud Firestore:

COLECCIÓN FIRESTORE	PATRÓN DE AISLAMIENTO	PROPÓSITO OPERATIVO
`resources`	`businesses/{id}/resources`	Contiene la definición física de la mesa: `capacity`, `minSpendDefault`, `activeHoldId`, `holdExpiresAt`.
`reservations`	`businesses/{id}/reservations`	Reservas activas del evento actual: `status`, `customerName`, `guests_count`, `deposit_amount`.
`attendanceSessions`	`businesses/{id}/attendanceSessions`	Sesiones de puerta generadas por el escáner del guardia: `state` (`INSIDE`, `OUTSIDE_TEMPORARY`, `ABSENT`).

COLECCIÓN FIRESTO-RE	PATRÓN DE AISLAMIENTO	PROPÓSITO OPERATIVO
`tableTabs`	`businesses/{id}/tableTabs`	Comandas y estado contable de consumo en mesa: `subtotal`, `items`, `status` (`OPEN` o `SETTLED`).

2.1 Algoritmo de Derivación Determinística de Estado La función pura `deriveTableOperationalState(...)` recibe las 4 fuentes y resuelve el estado atómico de cada mesa bajo la siguiente jerarquía estricta:

```
1. SI tableTab.status == 'SETTLED'                          ===> 'SETTLED' (Morado)
2. SI attendanceSession.state IN ['INSIDE', 'OUTSIDE_TEMPORARY'] ===> 'INSIDE' (Verde Esmeralda)
3. SI reservation.status IN ['CONFIRMED', 'PAID']           ===> 'CONFIRMED' (Dorado VIP)
4. SI activeHoldId ACTIVO Y holdExpiresAt > Date.now()      ===> 'HELD' (Ámbar 12 min)
5. EN CUALQUIER OTRO CASO                                   ===> 'AVAILABLE' (Gris Translúcido)
```

Este algoritmo garantiza que un cambio en puerta (check-in de reserva) pinte de inmediato la mesa en verde sin requerir recargar la página ni consultar múltiples bases de datos.

SECCIÓN 3 Sistema de Concierge y Cola de Peticiones VIP

El servicio de concierge (`conciergeService.ts`) permite la interacción asíncrona bidireccional entre el cliente en mesa y la brigada de meseros y corredores.

3.1 Estructura del Documento `ConciergeRequestRecord` Cada solicitud se persiste en la subcolección `businesses/{businessId}/conciergeRequests/{requestId}`:

```
{
  "requestId": "req_8849f1",
  "businessId": "club_barahunda",
  "uid": "usr_client_9941",
  "customerName": "Carlos Mendoza",
  "customerPhone": "+593991234567",
  "reservationId": "res_evt_sabado_vip_03",
  "resourceId": "table_vip_03",
  "tableCode": "VIP-03",
  "zone": "Palco Presidencial",
  "type": "BOTTLE_SERVICE | WAITER_CALL | EXTRA_ICE | SPARKLERS | BILL",
  "message": "Por favor 1 Don Pérignon Brut con bengala y 4 copas extra",
  "status": "PENDING | IN_PROGRESS | DONE | CANCELLED",
  "assignedWaiterUid": "staff_uid_mozo_12",
  "source": "in_app",
  "createdAt": "2026-10-12T23:45:10.200Z",
  "updatedAt": "2026-10-12T23:46:02.110Z"
}
```

3.2 Algoritmo de Priorización y Despacho en Tiempo Real La función `subscribeVenueConciergeRequests` aplica ordenamiento reactivo en memoria:

- **Criterio Primario:** Severidad del estado (`PENDING` [prioridad 0] > `IN_PROGRESS` [prioridad 1] > `DONE` [prioridad 2]).
- **Criterio Secundario:** Timestamp FIFO (`createdAt` ascendente para solicitudes pendientes, garantizando que el cliente que esperó más tiempo sea atendido primero).

```
const rank = (status: string) => {
  if (status === 'PENDING') return 0;
  if (status === 'IN_PROGRESS') return 1;
  return 2;
};
list.sort((a, b) => {
  const diffRank = rank(a.status) - rank(b.status);
  if (diffRank !== 0) return diffRank;
  return new Date(a.createdAt).getTime() - new Date(b.createdAt).getTime();
});
```

> **SLA RECOMENDADO PARA OPERACIONES VIP**
>
> - Peticiones `WAITER_CALL` (Llamar mesero): SLA p95 < 90 segundos.
> - Peticiones `EXTRA_ICE` / `SPARKLERS`: SLA p95 < 120 segundos.
> - Peticiones `BOTTLE_SERVICE`: Despacho en barra y mesa p95 < 180 segundos.

SECCIÓN 4 Máquina de Estados de Presencia y Control de Puerta (AGENTS §6)

El estado visual de la mesa en el mapa responde directamente al flujo de control de acceso en puerta:

```
ABSENT ──(Check-In inicial en puerta)──> INSIDE ──(Salida Temporal)──> OUTSIDE_TEMPORARY
──(Reingreso con Passkey)──> INSIDE
```



4.1 Pase Criptográfico Rotativo JWS (Ed25519) De conformidad con AGENTS §5, el QR escaneado en puerta es un token JWS firmado por Cloud Run / Cloud KMS con clave asimétrica:

- Rotación en Cliente: El token expira cada 45 segundos con refresco de 30 segundos.
- Prevención de Pantallazo: Si una persona toma captura de pantalla y se la reenvía a un amigo por mensajería, el token habrá expirado antes de que el receptor llegue a la fila de puerta.

4.2 Protocolo de Reingreso Anti-Transferencia por Passkey (AGENTS §6.2) Cuando un cliente en sala sale temporalmente (al auto o a fumar):

1. El portero escanea su pase y lo marca como `OUTSIDE_TEMPORARY`.
2. Se inicia una ventana de reingreso estricta de 30 minutos (`reentryExpiresAt`).
3. La PWA del cliente emite un desafío WebAuthn / Passkey vinculado al enclave de seguridad biométrico del smartphone original.
4. Para reingresar, el cliente presenta el pase y el dispositivo debe firmar con la Passkey de hardware. **Una captura reenviada a otra persona fallará la validación de hardware y será rechazada de inmediato.**

SECCIÓN 5 Ciclo de Vida de Holds (12 Min) y Liquidación de Comandas

5.1 Ciclo de Inventario y Holds (AGENTS §7)

```
AVAILABLE ──(Hold temporal 12 min)──> HELD ──(Inicio pasarela)──> PAYMENT_PENDING ──(Webhook OK)──>
CONFIRMED
                                        │                                  │
                                        └──(Timeout / Abandono)────────────┴──(Fallo pasarela)──>
AVAILABLE
```

1. Duración: 12 minutos exactos desde la creación del hold en el backend.
2. Liberación Concurrente:
   - **Modo Blaze:** Cloud Tasks programa la liberación al segundo exacto.
   - **Modo Híbrido Spark:** Un ticker interno `setInterval` (8 s) en el servidor de Render barre los documentos vencidos, complementado con la evaluación lazy transaccional en el momento de consulta.
3. Cero Sobreventa: Si un webhook llega tras expirar el hold y la mesa fue tomada por un tercero, el sistema crea automáticamente saldo a favor o reembolso; **nunca doble asignación**.

5.2 Procedimiento de Liquidación `settleTableTab` Cuando el cliente solicita la cuenta final, el staff con privilegios ejecuta la Cloud Function invocable `settleTableTab`:

Payload de Invocación:

```
callFunctions('settleTableTab', {
  businessId: activeClub.id,
  reservationId: selectedOp.activeReservationId,
  settlementMethod: 'cash' | 'card_manual' | 'transfer' | 'other',
  notes: 'Cierre de cuenta caja 2 - Mesa VIP-03 - Voucher 4921'
});
```

Lógica Transaccional en Servidor:

1. Verifica autenticación y claims del staff (`admin | manager | cashier`).
2. Lee el documento `businesses/{businessId}/tableTabs/{reservationId}` en transacción Firestore.
3. Calcula: `saldoCobrado = max(0, consumoTotal - anticipoDepositado)`.
4. Actualiza `tableTabs` a `status: 'SETTLED'`, registra timestamp y staffUid.
5. El listener de `operationsService` propaga el cambio en < 200 ms y la mesa pasa a color Morado Real en el plano de todos los dispositivos conectados.

SECCIÓN 6 Seguridad Multi-Tenant y Reglas de Integridad (AGENTS §2)

6.1 Regla de Aislamiento Innegociable Ningún documento de operaciones ni ticket de concierge puede ser consultado por un usuario que no pertenezca explícitamente al `businessId` solicitante:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /businesses/{businessId}/tableTabs/{tabId} {
      allow read: if request.auth != null && (
        request.auth.uid == resource.data.customerId ||
        exists(/databases/$(database)/documents/businesses/$(businessId)/staff/$(request.auth.uid))
      );
      allow write: if false; // Solo modificable mediante Cloud Functions transaccionales
    }
  }
}
```

6.2 Token Claims Mínimos (< 300 Bytes) Para evitar sobrecostes de cabecera HTTP y problemas de caché en proxies, el token JWT de Firebase Auth solo contiene alcance global (`platformRole` y `claimsVersion`). El `businessId` y los roles de staff se resuelven contra Firestore en cada operación.

SECCIÓN 7 Tolerancia a Fallos, Red Local y Contingencia Offline

7.1 Aislamiento de Red Total en Discoteca (AGENTS §6.3) En escenarios donde el proveedor de fibra óptica colapsa durante la noche:

1. **Caché Local IndexedDB:** Las terminales de puerta y sala almacenan en IndexedDB local el conjunto de claves públicas para validación criptográfica offline.
2. **NIGHTFLOW Edge (Gateway Local):** En establecimientos con servidor local, un nodo Edge coordina el aforo mediante WebSocket en LAN sin depender de internet.
3. **Resolución Idempotente de Colisiones:** Si dos puertas offline admiten el mismo pase antes de sincronizar, al recuperar internet Cloud Run detecta la duplicidad por `jti`, registra un documento en `businesses/{businessId}/securityIncidents` y alerta a la pantalla de gerencia. **Nunca se cobra ni se penaliza automáticamente al cliente; pasa a auditoría manual del administrador.**

7.2 Matriz Comparativa: Modo Híbrido Spark vs Modo Blaze Completo

COMPONENTE	MODO HÍBRIDO SPARK ($0/MES)	MODO BLAZE ENTERPRISE (PRODUCCIÓN)
Backend & API	Servidor Express único en Render Free (`node lib/server.js`)	Cloud Functions 2nd Gen en Google Cloud Platform
Módulo de Puerta	Montado en el mismo proceso bajo `/door/*`	Microservicio dedicado en Cloud Run (Latencia p95 < 300 ms)
Liberación de Holds	Ticker interno `setInterval` (8s) + verificación lazy	Google Cloud Tasks al segundo exacto
Gestión de Claves	Variables de entorno seguras en Render (`KEY_ED25519`)	Google Cloud KMS con firma asimétrica en hardware HSM

SECCIÓN 8 Matriz de Incidencias Operativas (Escenario 02:00 AM)

SÍNTOMA EN PANTALLA	CAUSA TÉCNICA PROBABLE	ACCIÓN CORRECTIVA INMEDIATA
Mesa en verde pero el cliente no está en la mesa	Check-in realizado en puerta pero el cliente está en barra o baño.	Verificar con el capitán si el titular fue guiado a su mesa. La presencia física en el club es válida.
Mesa en naranja (Hold) no se libera pasados 12 min	Retraso en ticker de background o cliente con modal de pasarela abierto.	El sistema bloqueará la compra en el checkout si expiró. Al recargar la vista se fuerza el recálculo lazy.
Error al pulsar "Cerrar Cuenta"	El usuario logueado no posee el rol `admin`, `manager` o `cashier`.	Solicitar al gerente o administrador de caja que inicie sesión en la terminal para liquidar.
Solicitud Concierge no cambia a "En atención"	Falla temporal de conexión WebSocket a Cloud Firestore.	Verificar conectividad Wi-Fi de la tablet. Al recuperar señal, el estado se sincroniza automáticamente.

Confidencial · Propiedad de Ingeniería & Operaciones de Plataforma